Project Technical Specification & Viva / Presentation Cheat Sheet

A concise overview of the technologies, algorithms, libraries, and design choices used in the Sungrow Web Application.

1. Quick Project Summary (Elevator Pitch)

> *"This is a full-stack enterprise web application replicating the Sungrow platform. It is built using **Next.js 16 (App Router)** and **React 19** on the frontend, with **Tailwind CSS v4** and **Motion** for animations, **Redux Toolkit & RTK Query** for state and API caching, and a secure authentication engine using **Argon2**, **JWTs in httpOnly cookies**, and **HMAC verification codes**."*

2. Tech Stack & Software Specifications

Layer	Technology	Why It Was Used
Framework	**Next.js 16 (Turbopack)**	Server-side rendering (SSR), optimized App Router, built-in API route handlers, high speed.
UI Library	**React 19**	Latest React features, Server Components, fast DOM rendering.
Language	**TypeScript 5 (Strict)**	Static typing, compile-time error checking, robust code quality.
Styling	**Tailwind CSS v4**	Utility-first, zero-runtime CSS, fully responsive design, dark/light themes.
State Management	**Redux Toolkit + RTK Query**	Centralized global store, automatic API caching, silent re-authentication, tag invalidation.
Forms & Validation	**React Hook Form + Zod**	Controlled form state without re-render lag, schema-based client & server validation.
Animations	**Motion (Framer Motion v14)**	Scroll reveals, curtain transitions, smooth mobile drawer slide-ins.
Sliders/Carousels	**Embla Carousel React**	Touch-friendly, lightweight, high-performance carousel with autoplay.
Icons	**Lucide React**	Clean, modern SVG icon set with tree-shaking support.
Security & Auth	**Argon2 + Jose (JWT) + HMAC**	Industry-standard password hashing and cryptographically secure tokens.
Testing	**Vitest + Playwright**	Vitest for fast unit tests; Playwright for cross-browser end-to-end automation.

3. Packages & Libraries: What Each One Does

- `next` / `react` / `react-dom`: The core React web application and App Router server.
- `@reduxjs/toolkit` & `react-redux`: Global state (`authSlice`, `uiSlice`, `registerSlice`) and `authApi` for backend endpoints.
- `motion`: Handles UI entrance animations, sticky scroll triggers, and tab transitions.
- `embla-carousel-react` & `embla-carousel-autoplay`: Powers the responsive image and card sliders (e.g., Sustainability reports, milestone timelines).
- `react-hook-form` & `@hookform/resolvers`: Manages form inputs, submit events, and validation errors cleanly without lag.
- `zod`: Declares validation schemas for email, passwords, login forms, and server payloads.
- `@node-rs/argon2`: High-speed native Argon2 password hasher.
- `jose`: Modern, lightweight JWT signing and verification library with zero external dependencies.
- `lucide-react`: Supplies navigation and status icons (Search, User, Chevron, Check, Alert, etc.).
- `clsx` & `tailwind-merge`: Safely combines dynamic Tailwind CSS class names without collisions.

4. Key Algorithms & Cryptographic Methods

1. **Password Hashing (Argon2id):**
   Method: `@node-rs/argon2`
   Why: Argon2 is the winner of the Password Hashing Competition. It is memory-hard, making it highly resistant to GPU and ASIC brute-force attacks.
2. **Session Authentication (Dual-Token JWT HS256):**
   Short-lived Access Token (15 min): Carries user identity in memory/cookies for fast stateless API checks.
   Long-lived Refresh Token (30 days): Used to obtain new access tokens without requiring re-login.
   Storage: Stored in `httpOnly`, `SameSite=Lax`, `Secure` cookies to prevent XSS (Cross-Site Scripting) token theft.
3. **OTP / Verification Code Security (HMAC-SHA256):**
   Method: `crypto.createHmac('sha256', secret)`
   How: Verification codes are combined with the user email and purpose, hashed with HMAC before storing, preventing plain-text exposure and timing attacks.
4. **Silent Refresh with Mutex Pattern (`baseQueryWithReauth`):**
   Problem: If multiple API calls fail with 401 at the same time, we shouldn't send multiple simultaneous refresh requests.
   Solution: A Mutex lock holds concurrent requests until a single refresh request completes, then replays the failed requests transparently.

5. Page-by-Page Feature Map

- **Home (`/`):** Hero scroll video sequence with pinned curtain transitions, scenario tabs, interactive solar product displays, news highlights.
- **Brand Story (`/about-us/brand-story`):** Interactive milestones carousel, full-bleed media banner, corporate footprint stats.

- **C&I PV Solution (`/solution/ci-pv-solution`):** Commercial & Industrial solar energy solutions, system architecture breakdown, contact lead form.
- **Residential Solution (`/solution/residential-pv-ess-ev-charging-solution`):** Residential PV + Energy Storage + EV Charging ecosystem showcase.
- **Sustainability (`/sustainability`):** ESG pillars (Environmental, Social, Governance), climate targets, ESG report download cards.
- **Auth Flow (`/login` & `/register`):** Multi-tab login (Password or OTP verification code), multi-step registration flow (Country → Role → Email → Password), dialogs for Password Reset, Delete Account, and Logout.

6. Likely Invigilator Cross-Questions & Quick Answers

Q1: "Why did you choose Next.js App Router instead of standard React (Vite/CRA)?"

Answer: "Next.js App Router gives us Server-Side Rendering (SSR) for fast initial page loads and SEO optimization, plus built-in API Route handlers so we don't need a separate Express backend server."

Q2: "How is user authentication handled?"

Answer: "Using JWTs stored in `httpOnly` cookies. Access tokens expire after 15 minutes, while refresh tokens last 30 days. When an access token expires, our custom RTK Query base query triggers a silent refresh using a mutex pattern so the user stays logged in seamlessly."

Q3: "Why Argon2 instead of bcrypt for password hashing?"

Answer: "Argon2 is the modern industry standard and winner of the Password Hashing Competition. It provides superior memory-hardness against modern GPU-accelerated cracking compared to older algorithms like bcrypt or MD5."

Q4: "How do you prevent state lag on large forms?"

Answer: "We use `react-hook-form`, which utilizes uncontrolled inputs with React refs to prevent unnecessary component re-renders on every keystroke, coupled with `zod` for strict schema validation."

Q5: "Where is the data stored?"

Answer: "For this implementation, the data store uses an atomic JSON store (`store.ts`) on the server side to persist users and sessions locally without requiring external cloud database dependencies."